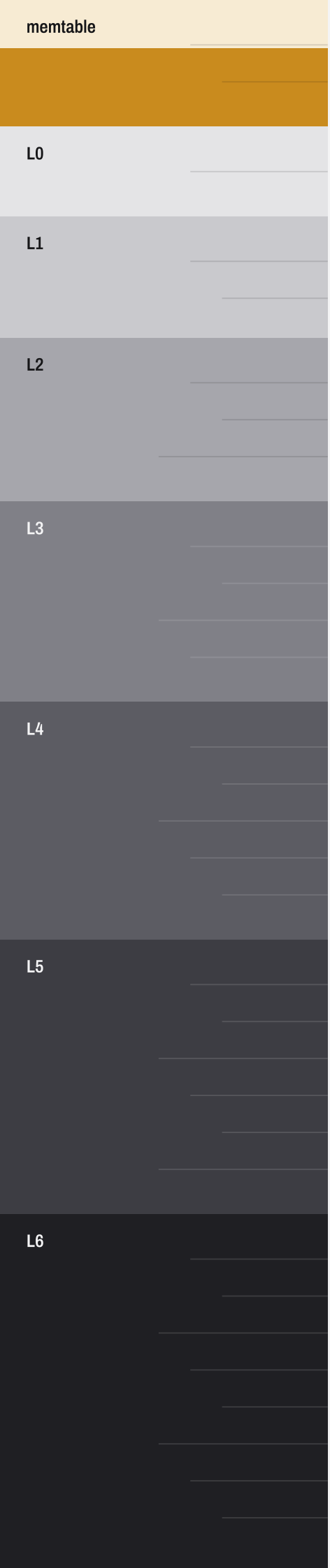




Banco de dados LSM-Tree

Arca

Arquitetura do banco de dados

Como a Arca organiza e grava os dados em disco. LSM-Tree, formatos binários do WAL, das SSTables e do MANIFEST, codificação de chaves e linhas, compactação, recuperação, checkpoints e verificação de integridade. É a arquitetura que a aplicação segue e que qualquer extensão deve respeitar.



Memtable: escritas recentes, em memória

SSTables nos níveis L0 a L6, em disco, descendo pela compactação

Versão 1.2.1

Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 **Visão da arquitetura** 4

1.1 Caminho de uma escrita 4

1.2 Camadas 5

02 **Arquivos no diretório** 6

2.1 Tipos de arquivo 6

03 **Chave interna e sequência** 7

3.1 Formato 7

3.2 Ordenação 7

04 **Log de escrita (WAL)** 8

4.1 Fragmento físico 8

4.2 Conteúdo de um registro: WriteBatch 8

4.3 Group commit 8

05 **Memtable** 9

5.1 Entrada 9

5.2 Troca da memtable 9

06 **SSTable** 10

6.1 Layout de um arquivo .sst 10

6.2 Bloco de dados 10

6.3 Índice, Bloom e propriedades 10

6.4 Cache 11

07 **MANIFEST e versões** 12

7.1 VersionEdit 12

7.2 Troca do MANIFEST 12

08 **Compactação** 13

8.1 Gatilhos 13

8.2 O que é descartado 13

09 **Recuperação após queda** 14

9.1 Passo a passo 14

10 **Modelo lógico: chaves, tabelas e índices** 15

Copyright (©) 2026 Erik Pimenta

10.1 Espaços de chave 15
10.2 Codificação ordenável 15
10.3 Linha 15
10.4 Tabelas criadas pela aplicação 16

11 Concorrência 17

11.1 Travas por recurso 17

12 Preparando o armazenamento 18

12.1 O que vale preparar 18
12.2 Verificação 18

13 Checkpoints e verificação de integridade 19

13.1 Como um checkpoint é criado 19
13.2 ARCA.CHECKPOINTS 19
13.3 ARCA.VERIFY 20

Copyright (©) 2026 Erik Pimenta

01 Visão da arquitetura

A Arca usa uma Log-Structured Merge-Tree. Nenhum dado é alterado no lugar: cada mudança é acrescentada e versões antigas somem quando os arquivos são mesclados.

1.1 Caminho de uma escrita

O diagrama mostra o caminho de uma escrita, do lote até os níveis mais profundos. A memória aparece em âmbar e o disco em tons de grafite, do mesmo jeito que na coluna de estratos do painel.



Escritas são sequenciais no WAL e na memtable; os arquivos em disco são ordenados e imutáveis, organizados em níveis, e a compactação desce os dados nível a nível.

Copyright (©) 2026 Erik Pimenta

1.2 Camadas

Camada	Onde fica	Papel
WAL	`NNNNNN.wal`	Garante que nada confirmado se perca antes de virar SSTable
Memtable	memória	Skip list ordenada com as escritas recentes; leitura sem trava
Memtable imutável	memória	A memtable cheia, aguardando a gravação em disco
SSTables	`NNNNNN.sst`	Arquivos ordenados, imutáveis, com índice, Bloom e checksums
MANIFEST	`MANIFEST-NNNNNN`	Log das versões: quais SSTables existem em cada nível
CURRENT	`CURRENT`	Nome do MANIFEST em uso, trocado de forma atômica
Checkpoints	`checkpoints/nome/`	Cópias consistentes do banco, ao lado da pasta `dir` (capítulo 13)

Copyright (©) 2026 Erik Pimenta

02 Arquivos no diretório

Cada banco é uma pasta. Os números de arquivo vêm de um único contador, então nunca se repetem.

2.1 Tipos de arquivo

Arquivo	Exemplo	Conteúdo	Quando é apagado
CURRENT	`CURRENT`	Uma linha com o nome do MANIFEST atual	Nunca (é substituído por renomeação)
LOCK	`LOCK`	Trava do sistema operacional (fcntl ou abertura exclusiva no Windows)	Nunca
MANIFEST	`MANIFEST-000005`	Registros de VersionEdit no formato do log	Quando um MANIFEST mais novo assume
WAL	`000007.wal`	Lotes de escrita (WriteBatch) no formato do log	Depois que a memtable dele vira SSTable
SSTable	`000012.sst`	Tabela ordenada e imutável	Quando nenhuma versão viva a referencia
Temporário	`000005.tmp`	Arquivo do CURRENT antes da renomeação	Logo após a troca

- Um arquivo só é removido quando não aparece em nenhuma versão ainda em uso. Iteradores e snapshots seguram a versão que estavam lendo, então uma varredura longa nunca perde arquivos no meio do caminho. Durante um checkpoint ou uma verificação de integridade as remoções ficam suspensas, para que nenhuma SSTable suma antes de ser ligada ao checkpoint ou conferida.

Copyright (©) 2026 Erik Pimenta

03 Chave interna e sequência

Toda escrita recebe um número de sequência de 56 bits que só cresce. A chave guardada no motor é a chave interna: a chave do usuário seguida de 8 bytes que juntam a sequência e o tipo da operação.

3.1 Formato



8 bytes little-endian no fim: `(sequência << 8) | tipo`

Tipo	Valor	Significado
`kTypeDeletion`	0	Lápide: a chave foi apagada nesta sequência
`kTypeValue`	1	Valor gravado nesta sequência

3.2 Ordenação

A ordenação compara primeiro a chave do usuário em bytes e, para a mesma chave, a sequência decrescente. Assim a versão mais nova aparece primeiro e uma busca por (chave, sequência S) cai exatamente na versão mais nova visível em S. É isso que dá snapshots MVCC de graça.

Copyright (©) 2026 Erik Pimenta

04 Log de escrita (WAL)

O log é dividido em blocos de 32 KB. Cada registro lógico é quebrado em fragmentos físicos que nunca atravessam um bloco; um resto de bloco menor que o cabeçalho é preenchido com zeros.

4.1 Fragmento físico

Campo	Tamanho	Descrição
checksum	4 bytes	CRC32C mascarado do tipo + dados
tamanho	2 bytes	Bytes de dados deste fragmento
tipo	1 byte	`FULL` (1), `FIRST` (2), `MIDDLE` (3), `LAST` (4)
dados	variável	Pedaço do registro

4.2 Conteúdo de um registro: WriteBatch

Campo	Codificação
sequência inicial	fixo de 8 bytes
quantidade de operações	fixo de 4 bytes
cada operação	tipo (1 byte) + chave com prefixo de tamanho (varint) + valor com prefixo de tamanho, se for escrita

A operação *i* do lote recebe a sequência inicial + *i*. Um registro com checksum errado ou cortado no fim do arquivo é descartado na recuperação; o restante do log continua valendo. Com `paranoid-checks` ligado, qualquer corrupção impede a abertura.

4.3 Group commit

Escritores concorrentes entram numa fila. O primeiro vira líder, junta os lotes de quem está esperando (até 1 MB), grava um único registro e, se preciso, faz um único `fsync`. Os demais acordam com o resultado pronto. Com `appendfsync always`, isso divide o custo do `fsync` entre todos.

Copyright (©) 2026 Erik Pimenta

05 Memtable

A memtable é uma skip list com até 12 níveis (probabilidade 1/4 de subir), alocada numa arena de blocos de 4 KB.

5.1 Entrada

Só um escritor insere por vez (o líder do group commit); leitores percorrem a lista sem trava, protegidos por ponteiros atômicos com ordem acquire/release.

Parte da entrada	Codificação
tamanho da chave interna	varint32
chave interna	chave do usuário + 8 bytes de sequência e tipo
tamanho do valor	varint32
valor	bytes

5.2 Troca da memtable

Quando o uso da arena passa de `write-buffer-size`, a memtable vira imutável, um WAL novo começa e a thread de fundo grava a SSTable. Se ainda houver uma memtable imutável pendente, novas escritas esperam (travamento de escrita). O painel mostra esse ciclo na faixa âmbar do topo da coluna de estratos, que enche até o limite e esvazia a cada flush.

Copyright (©) 2026 Erik Pimenta

06 SSTable

Uma SSTable é um arquivo ordenado e imutável com blocos de dados, filtro de Bloom, propriedades, índice e um rodapé de tamanho fixo.

6.1 Layout de um arquivo .sst



Cada bloco termina com 1 byte de compressão (0 = nenhuma) e 4 bytes de CRC32C. O rodapé tem 68 bytes: referências (offset, tamanho) de Bloom, propriedades e índice + número mágico `0x4152434153535442`.

6.2 Bloco de dados

Blocos de cerca de 4 KB com entradas ordenadas. Para economizar espaço, cada chave guarda só a parte que difere da anterior; a cada 16 entradas há um ponto de reinício com a chave completa, usado na busca binária dentro do bloco.

Campo da entrada	Codificação
bytes compartilhados com a chave anterior	varint32
bytes novos da chave	varint32
tamanho do valor	varint32
parte nova da chave	bytes
valor	bytes
(fim do bloco) pontos de reinício	fixo de 4 bytes cada + quantidade em 4 bytes

6.3 Índice, Bloom e propriedades

- **Índice:** um bloco com uma entrada por bloco de dados; a chave é a última chave interna do bloco e o valor é (offset, tamanho) em varint.
- **Filtro de Bloom:** um filtro por arquivo sobre as chaves do usuário, com 10 bits por chave e 6 funções de hash (dupla dispersão), cerca de 1% de falso positivo. Uma leitura pontual consulta o filtro antes de tocar no disco.
- **Propriedades:** entradas, lápides, blocos, bytes de chave e de valor, menor e maior sequência, data de criação e bits do Bloom. O `arca-inspect` mostra todas.

Copyright (©) 2026 Erik Pimenta

6.4 Cache

Blocos lidos vão para um cache LRU dividido em 16 fatias, cada uma com seu mutex, limitado por `block-cache-size`. Arquivos abertos ficam num cache de tabelas protegido por `shared_mutex`: consultas usam trava compartilhada e só a abertura de um arquivo novo usa trava exclusiva. Toda leitura de bloco que vem do disco confere o CRC32C antes de entrar no cache.

Copyright (©) 2026 Erik Pimenta

07 MANIFEST e versões

Uma versão é a lista de SSTables de cada nível. Versões são imutáveis e compartilhadas por ponteiro com contagem de referências.

7.1 VersionEdit

Toda mudança (flush, compactação) é descrita por um VersionEdit, gravado no MANIFEST com `fsync` antes de a nova versão entrar em uso.

Marcador	Campo	Codificação
2	número do log em uso	varint64
3	próximo número de arquivo	varint64
4	última sequência	varint64
5	ponteiro de compactação	nível + chave interna
6	arquivo removido	nível + número
7	arquivo adicionado	nível, número, tamanho, entradas, lápides, menor e maior chave

7.2 Troca do MANIFEST

Ao abrir o banco, um MANIFEST novo é criado com o retrato completo da versão atual; depois o CURRENT passa a apontar para ele por renomeação atômica e o antigo é apagado. O `arca-inspect manifest` mostra cada edição em texto.

Copyright (©) 2026 Erik Pimenta

08 Compactação

A compactação mistura arquivos de um nível com os do nível de baixo, descarta o que ninguém mais enxerga e mantém cada nível dentro do limite.

8.1 Gatilhos

Nível	Gatilho	Regra
L0	4 arquivos (pontuação = arquivos ÷ 4)	Arquivos se sobrepõem; todos os que tocam o intervalo entram juntos
L1	mais de 10 MB	Arquivos não se sobrepõem; um arquivo por vez, em rodízio pelo ponteiro de compactação
L*k* (*k* ≥ 2)	mais de 10 MB × 10^{k-1}	Igual ao L1

A thread de fundo escolhe o nível com maior pontuação maior que 1, junta os arquivos escolhidos com os que se sobrepõem no nível de baixo e grava SSTables novas de até `max-file-size`, cortando também quando a saída se sobreporia demais ao nível seguinte. Um arquivo que não se sobrepõe a nada é só movido de nível, sem reescrita.

8.2 O que é descartado

1. Versões antigas de uma chave cuja versão mais nova já é visível para todos os snapshots abertos.
2. Lápides com sequência visível para todos, quando nenhum nível mais profundo pode ter a chave.

Snapshots seguram versões: a menor sequência de snapshot aberto é o limite abaixo do qual a compactação pode limpar. Os travamentos de escrita começam com 8 arquivos no L0 (pausa de 1 ms por escrita) e as escritas param com 12, até a compactação alcançar.

`ARCA.COMPACT` grava a memtable e compacta cada nível inteiro para o de baixo até o nível mais profundo que já tem dados, o que remove lápides e versões antigas de todo o banco.

Copyright (©) 2026 Erik Pimenta

09 Recuperação após queda

Na abertura, o banco reconstrói a última versão confirmada e reaplica o que ainda estava só no log.

9.1 Passo a passo

1. Trava o diretório pelo arquivo `LOCK`.
2. Lê o CURRENT e reaplica cada VersionEdit do MANIFEST, reconstruindo a versão, o próximo número de arquivo e a última sequência.
3. Confere que todas as SSTables citadas existem; se faltar alguma, a abertura falha e informa qual.
4. Reaplica, em ordem, os WALs com número maior ou igual ao log registrado. Fragmentos com checksum errado ou cortados no fim são descartados.
5. Grava o que foi reaplicado como SSTables do L0, cria um WAL novo e um MANIFEST novo.
6. Apaga logs, MANIFESTs e SSTables que não pertencem a nenhuma versão viva.

- Como as SSTables são imutáveis e só passam a valer depois do `fsync` do MANIFEST, uma queda no meio de um flush ou de uma compactação deixa apenas arquivos órfãos, que são apagados na próxima abertura.

Copyright (©) 2026 Erik Pimenta

10 Modelo lógico: chaves, tabelas e índices

Chave-valor e SQL convivem no mesmo espaço de chaves, separados pelo primeiro byte. Como a ordenação é por bytes, cada espaço ocupa uma faixa contínua e pode ser varrido sem tocar nos outros.

10.1 Espaços de chave

Prefixo	Espaço	Chave	Valor
`0x01`	chave-valor	`0x01` + chave	flag (1 byte) + [expiração em ms, 8 bytes] + valor
`0x02`	catálogo	`0x02` + nome da tabela	definição: id, chave primária, colunas, índices
`0x03`	linhas	`0x03` + id da tabela (4 bytes big-endian) + chave primária ordenável	linha codificada
`0x04`	índices	`0x04` + id do índice + marcador de nulo + valor ordenável + chave primária	vazio
`0x05`	contadores	`0x05` + nome	inteiro de 8 bytes (próximo id, autoincremento, linhas)
`0x06`	expiração	`0x06` + instante (8 bytes big-endian) + chave	vazio

10.2 Codificação ordenável

Tipo	Bytes	Por que ordena
`INT`	8 bytes big-endian com o bit de sinal invertido	Negativos ficam antes dos positivos
`REAL`	8 bytes IEEE 754: positivos com o sinal ligado, negativos com todos os bits invertidos	A ordem dos bytes segue a ordem numérica
`TEXT`	bytes com `0x00` trocado por `0x00 0xFF` e terminador `0x00 0x01`	Prefixo vem antes da palavra maior
`BOOL`	1 byte (0 ou 1)	`FALSE` antes de `TRUE`

10.3 Linha

Quantidade de colunas em varint e, para cada coluna, um marcador de tipo (0 nulo, 1 inteiro em zigzag varint, 2 real de 8 bytes, 3 texto com tamanho, 4 booleano). Linhas antigas com menos colunas leem as que faltam como `NULL`, o que torna `ALTER TABLE ADD COLUMN` barato.

Copyright (©) 2026 Erik Pimenta

10.4 Tabelas criadas pela aplicação

clientes	
INT	id PK AUTOINCREMENT
TEXT	nome NOT NULL
TEXT	email
TEXT	cidade idx_clientes_cidade
BOOL	ativo DEFAULT TRUE
INT	criado_em

produtos	
INT	id PK AUTOINCREMENT
TEXT	sku NOT NULL
TEXT	nome NOT NULL
TEXT	categoria idx_produtos_categoria
REAL	preco
INT	estoque DEFAULT 0

pedidos	
INT	id PK AUTOINCREMENT
INT	cliente_id → clientes.id idx_pedidos_cliente
INT	produto_id → produtos.id
INT	quantidade
REAL	total
TEXT	status idx_pedidos_status
INT	criado_em

_painel_usuarios	
TEXT	email PK
TEXT	name · role · password
INT	created_at
INT	failed · locked_until DEFAULT 0

_painel_auditoria	
INT	id PK AUTOINCREMENT
INT	at
TEXT	email · action · detail

_painel_tokens	
TEXT	id PK
TEXT	hash índice · SHA-256
TEXT	name · role · owner · prefix
INT	created_at · last_used_at · expires_at

_painel_webhooks	
TEXT	id PK
TEXT	url · events · secret
INT	active DEFAULT 1
INT	created_at · last_status · last_delivery_at · failures

_painel_webhook_entregas	
INT	id PK AUTOINCREMENT
TEXT	webhook → _painel_webhooks.id índice
TEXT	delivery · event · error
INT	at · status
REAL	ms

_painel_consultas	
TEXT	id PK
TEXT	name · query · owner
INT	shared DEFAULT 0
INT	created_at

Tabelas de exemplo (criadas com `ARCA_SEED=true`, borda contínua) e tabelas de sistema do painel (borda tracejada). As setas são relações lógicas.

As relações entre `pedidos`, `clientes` e `produtos` são lógicas: a Arca não impõe chave estrangeira, e as consultas usam os índices `idx_pedidos_cliente` e `idx_pedidos_status`. Tabelas iniciadas por `_` são de sistema. As sessões do painel ficam no espaço chave-valor como `_painel:sessao:<sha256 do token>`, com TTL. As chaves de acesso da 1.2.0 seguem a mesma regra: o banco guarda só o SHA-256 da chave, nunca a chave em si.

Copyright (©) 2026 Erik Pimenta

11 Concorrência

Cada recurso compartilhado tem o seu mecanismo de trava, escolhido para que leituras quase nunca esperem por escritas.

11.1 Travas por recurso

Recurso	Mecanismo	Quem disputa
Fila de escrita	`std::mutex` + uma `condition_variable` por escritor (group commit)	Todos os comandos que gravam
Arquivo do WAL	`std::mutex` próprio	Líder da fila, thread de fsync, troca de memtable
Versão atual (SuperVersion)	`std::shared_mutex`: leitura compartilhada, troca exclusiva	Leituras e a thread de fundo
Memtable	Skip list com ponteiros atômicos (acquire/release)	Um escritor e vários leitores sem trava
Cache de blocos	16 fatias, cada uma com `std::mutex`	Leituras
Cache de tabelas	`std::shared_mutex`	Leituras e aberturas de arquivo
Catálogo SQL	`std::shared_mutex` global + um por tabela	DDL exclusivo; `SELECT` compartilhado; `INSERT` / `UPDATE` / `DELETE` exclusivos por tabela
Chaves	1024 mutex em faixas por hash da chave	`INCR`, `APPEND`, `SET NX/XX`, `SETRANGE`, `COPY`, `MSETNX`, expiração
Transações	`std::shared_mutex`: comandos compartilham, `EXEC` é exclusivo	`MULTI` / `EXEC`
Remoção de arquivos	Contador de suspensão protegido pelo mutex do banco	Checkpoint, `ARCA.VERIFY` e limpeza de arquivos obsoletos
Rede	Uma thread por laço de eventos; fila com mutex para conexões novas e respostas dos workers	Laços e workers

Copyright (©) 2026 Erik Pimenta

12 Preparando o armazenamento

A Arca não precisa de um servidor de banco à parte nem de script de criação: o diretório e os arquivos são criados na primeira abertura, e as tabelas surgem com `CREATE TABLE`.

12.1 O que vale preparar

- **Disco:** SSD é recomendado. Reserve pelo menos o dobro do tamanho dos dados, porque a compactação grava as SSTables novas antes de apagar as antigas.
- **Sistema de arquivos:** um com `fsync` confiável (ext4, xfs, APFS, NTFS). Evite pastas de rede compartilhadas.
- **Docker:** os dados ficam no volume `dados` (`/dados/banco` e `/dados/checkpoints`). O contêiner roda como usuário sem privilégios (uid 10001).
- **Permissões:** o usuário do processo precisa ler e gravar a pasta do banco e a pasta `checkpoints` ao lado dela.
- **Mesmo sistema de arquivos:** deixe `checkpoints/` no mesmo volume do banco, para que os checkpoints usem links físicos em vez de cópias.
- **Backup:** agende `ARCA.CHECKPOINT` e copie a pasta gerada para outro lugar; restaurar é apontar `dir` para ela.
- **Migração entre máquinas:** os formatos têm ordem de bytes definida e não dependem da plataforma; a pasta pode ser copiada entre Linux, macOS e Windows com o servidor parado.

12.2 Verificação

Verificação	Comando
Checksums das SSTables vivas, servidor no ar	`ARCA.VERIFY` ou **Verificar integridade** na tela Armazenamento
Checksums de todas as SSTables, servidor parado	`arca-inspect check dados`
Versão atual e arquivos por nível	`arca-inspect manifest dados/MANIFEST-NNNNNN`
Lotes ainda não gravados em SSTable	`arca-inspect wal dados/NNNNNN.wal`
Checkpoints existentes	`ARCA.CHECKPOINTS`
Estado ao vivo	`arca-cli ARCA.STATS`, `/metrics` ou a tela Armazenamento do painel

Copyright (©) 2026 Erik Pimenta

13 Checkpoints e verificação de integridade

Um checkpoint é um banco completo, consistente e pronto para abrir, criado sem parar o servidor. A verificação de integridade lê cada bloco das SSTables vivas e confere o checksum, também com o servidor no ar.

13.1 Como um checkpoint é criado

1. A memtable é gravada como SSTable (`Flush`), para que tudo o que foi confirmado esteja em arquivos imutáveis.
2. Sob o mutex do banco, a remoção de arquivos obsoletos é suspensa e a versão atual é capturada junto com a última sequência.
3. Cada SSTable da versão é ligada à pasta do checkpoint com um link físico; se o sistema de arquivos não permitir (outro volume, por exemplo), o arquivo é copiado.
4. Um MANIFEST novo é gravado na pasta com um único VersionEdit que descreve a versão inteira: arquivos por nível, última sequência, próximo número de arquivo e log 0 (não há WAL a reaplicar).
5. O `CURRENT` da pasta passa a apontar para esse MANIFEST, e a remoção de arquivos volta a funcionar.

```
dados/                          # banco em uso (dir)
├── CURRENT  LOCK  MANIFEST-000044
├── 000043.wal
└── 000031.sst  000038.sst  000042.sst
checkpoints/                    # ao lado de dir
└── antes-da-migracao/
    ├── CURRENT                 # aponta para o MANIFEST abaixo
    ├── MANIFEST-000043         # um VersionEdit com o retrato completo
    └── 000031.sst  000038.sst  000042.sst   # links físicos (ou cópias)
```

Estrutura de um checkpoint. O nome aceita letras, números, `.` , `-` e `_` ; sem nome, o servidor usa `checkpoint-<segundos desde 1970>` . O destino não pode existir.

Como os arquivos são imutáveis, o link físico é seguro: quando a compactação substitui uma SSTable no banco, ela só remove o nome dentro de `dados/` ; o conteúdo continua existindo enquanto o checkpoint apontar para ele. Por isso um checkpoint recém-criado ocupa quase nada, e passa a ocupar espaço próprio à medida que o banco compacta.

13.2 ARCA.CHECKPOINTS

Lista as subpastas de `checkpoints/` em JSON, da mais nova para a mais antiga, com o nome, a data de criação (ms), o total de bytes e a quantidade de arquivos. `ARCA.CHECKPOINTS DELETE nome` apaga uma delas. O nome é validado (só `[A-Za-z0-9._-]` , nunca `.` ou `..`), então o comando nunca sai da pasta de checkpoints, e só

Copyright (©) 2026 Erik Pimenta

apaga pastas que têm um `CURRENT`, ou seja, que são de fato bancos. No gateway, `GET /api/storage/checkpoints` lista (Leitura) e `DELETE /api/storage/checkpoints/{nome}` apaga (Administrador).

```
127.0.0.1:7379> ARCA.CHECKPOINTS
"[{\"name\":\"diario\",\"created_ms\":1793110156760,\"bytes\":3718204,\"files\":8}]"
```

Para restaurar, pare o servidor e aponte `dir` para a pasta do checkpoint (ou copie o conteúdo dela para `dados/`). Na abertura ele passa pela recuperação normal do capítulo 9: lê o CURRENT, reconstrói a versão e cria WAL e MANIFEST novos.

13.3 ARCA.VERIFY

Comando pesado (vai para o pool de workers e não pode ser usado dentro de `MULTI`) que percorre todas as SSTables da versão atual, nível por nível:

1. Sob o mutex do banco, suspende a remoção de arquivos obsoletos (o mesmo contador usado pelo checkpoint) e captura a versão atual.
2. Abre cada SSTable direto do disco, sem passar pelo cache de tabelas nem pelo cache de blocos. A abertura já lê o rodapé e confere o número mágico, e lê o índice, as propriedades e o filtro de Bloom com checksum.
3. Percorre todos os blocos de dados. Cada bloco tem o CRC32C recalculado sobre os dados e o byte de compressão e comparado com os 4 bytes gravados no fim do bloco.
4. Confere que as chaves internas aparecem em ordem estritamente crescente, como na verificação do `arca-inspect`.
5. Conta arquivos e bytes; um arquivo com qualquer falha entra na lista `corrupt` com o nome e a mensagem do erro. No fim, a remoção de arquivos volta a funcionar.

```
{"files":6,"bytes":3718204,"ok":6,"corrupt":[],"ms":41.7}

{"files":6,"bytes":3718204,"ok":5,
 "corrupt":[{"file":"000042.sst","error":"Corruption: checksum do bloco não
confere"}],"ms":40.2}
```

Escritas e compactações continuam durante a verificação; como a remoção fica suspensa, nenhum arquivo verificado é apagado no meio do caminho, e os que ficaram obsoletos são limpos logo depois. A verificação não confere o WAL nem o MANIFEST, que já são validados registro a registro na abertura; para isso, use `arca-inspect wal` e `arca-inspect manifest` com o servidor parado.

Copyright (©) 2026 Erik Pimenta